WEEK 2 / LESSON 1 / 50 MINUTES

L1

Python Data Structures

80% of "my pandas is slow" bugs are actually "I chose the wrong data structure below pandas". Pick the structure first; the algorithm is easy afterwards.

Question: how many milliseconds to check `item in my_list` across 1M items? And across `my_set`? We demo live.

WHAT'S INSIDE

WHY THIS LESSON MATTERS

Flipkart catalog: 4.2 million SKU lookups per second.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1. Choose the correct structure for a scenario in under 10 seconds.
2. Predict the complexity of common operations on each.
3. Convert fluently between the four structures.
4. Recognise the three most common Big-O surprises.
5. Apply comprehensions where they improve readability.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

list — the everyday...

Ordered, duplicates allowed

02

dict — O(1) lookup by key

Replace any 3+ if/elif ladder

03

set — membership and dedup

O(1) contains; set algebra built in

04

tuple — immutable twin of...

Dict keys, set elements, multi-return

01

PART 1 OF 4

list — the everyday...

Ordered, duplicates allowed

CONCEPT 1 OF 4

list — the everyday workhorse

Append O(1). Index access O(1). Membership O(n).
Slicing: a[::-1] is the reversed copy — O(n).
Insert at front O(n) — use collections.deque for frequent front-ops.
Sort is Timsort — O(n log n), stable.
USE WHEN
Order + duplicates

Iteration is frequent; lookup is rare.

DEFINITION

list — the everyday workhorse

Ordered, duplicates allowed

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

dict — O(1) lookup by key

Keys must be hashable: str, int, tuple — never list, never dict.
d.get(key, default) — safe access.
d.setdefault(key, []).append(v) — group-by in one line.
Dict comprehension: {k: v*2 for k, v in d.items()}.
USE WHEN
Key → Value

You need constant-time lookup.

KEY POINTS

- Keys must be hashable: str, int, tuple — never list, never dict
- d.get(key, default) — safe access
- d.setdefault(key, []).append(v) — group-by in one line
- Dict comprehension: {k: v*2 for k, v in d.items()}

CONCEPT 3 OF 4

set — membership and dedup

valid = set(ids) → `id in valid` is O(1).
a & b intersection; a | b union; a - b difference.
list(set(xs)) dedupes; dict.fromkeys(xs) dedupes AND preserves order.
Set comprehension: {x.lower() for x in xs}.
USE WHEN
Unique + fast "in"

Order does not matter.

DEFINITION

set — membership and dedup

O(1) contains; set algebra built in

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 4 OF 4

tuple — immutable twin of list

Immutable after creation.
Can be used as a dict key or set element.
Multi-return: return (name, age).
Named tuples give you fields — bridge to dataclasses.
USE WHEN
Fixed tuple of values

Think: coordinates, (year, month) keys.

KEY POINTS

- Immutable after creation
- Can be used as a dict key or set element
- Multi-return: return (name, age)
- Named tuples give you fields — bridge to dataclasses

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK · PAIR · SHARE

ACTIVITY

Pick the right structure for each scenario

PROMPT

Assign list / dict / set / tuple to: (a) seat allocations for a 10-row theatre (b) the last N tweets in chronological order (c) unique email addresses across 10 CSVs (d) (lat, lon) coordinates on a map.

HOW TO RUN IT

1. Assign list / dict / set / tuple to: (a) seat allocations for a 10-row theatre (b) the…
2. Debate disagreements. Each pair must defend one choice.
3. One pair presents their rationale to the room.
4. Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1 *Which structure has been your default — and is it always the right one?*

Q2 *Where did you last write an O(n) membership check that could be O(1)?*

Q3 *One habit you will adopt this week to think "structure first".*

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

KNOWLEDGE CHECK · QUESTION 1 OF 3

Which has O(1) membership?

A list

B dict

C tuple

D str

ANSWER · QUESTION 1 OF 3

Which has O(1) membership?

B

CORRECT ANSWER

dict

WHY

dict (and set) use hashing for O(1) lookup. list and tuple are O(n)

KNOWLEDGE CHECK · QUESTION 2 OF 3

Why can't a list be a dict key?

A Lists are slow

B Lists are mutable — not hashable

C Lists are indexed

D Python forbids it

ANSWER · QUESTION 2 OF 3

Why can't a list be a dict key?

B

CORRECT ANSWER

Lists are mutable — not hashable

WHY

Dict keys must be hashable and immutable. Lists are mutable; tuples are fine

KNOWLEDGE CHECK · QUESTION 3 OF 3

Dedup AND preserve order of [3,1,2,1]:

A set(xs)

B list(set(xs))

C list(dict.fromkeys(xs))

D sorted(set(xs))

ANSWER · QUESTION 3 OF 3

Dedup AND preserve order of [3,1,2,1]:

C

CORRECT ANSWER

list(dict.fromkeys(xs))

WHY

dict.fromkeys preserves insertion order (Python 3.7+)

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

Append O(1). Index access O(1). Membership O(n).
Slicing: a[::-1] is the reversed copy — O(n).
Insert at front O(n) — use collections.deque for frequent front-ops.
Sort is Timsort — O(n log n), stable.
USE WHEN
Order + duplicates

Iteration is frequent; lookup is rare.

THE TAKEAWAY

"80% of slow pandas code is a wrong-structure bug below pandas. Master the four; pay that cost never again."

WHAT TO NOTICE

1. list — the everyday…
2. dict — O(1) lookup by key
3. set — membership and dedup
4. tuple — immutable twin of…

APPLY IT → ASSIGNMENT

Mini assignment — replace one O(n) with O(1)

Take any Week-1 script you wrote. Find one list membership check inside a loop.

DELIVERABLES

- Convert the list to a set at the top of the function.
- Measure with %timeit before and after.
- Post the before/after timings + the diff in a cohort-forum thread.
- Document one decision you made because of this lesson.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Screenshot of the %timeit comparison + 2-sentence explanation.

ONE THING TO REMEMBER

> *"80% of slow pandas code is a wrong-structure bug below pandas. Master the four; pay that cost never again."*

— AI Learning Hub

END OF LESSON 2

What you can now do

Each one of these is now part of your working vocabulary.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

NEXT → Lesson 2 — NumPy Fundamentals. The speed layer under every Python ML library.